



ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
UNIVERSITY OF SCIENCE AND TECHNOLOGY - UD



CƠ SỞ DỮ LIỆU

TS. Võ Đức Hoàng

Khoa Công nghệ Thông tin

Trường Đại học Bách khoa - Đại học Đà Nẵng

THINKING - CREATING - HUMANITY CHERISHING





CHƯƠNG 12

Chỉ mục, tối ưu hóa truy vấn SQL với EXPLAIN, ACID / Transaction

NỘI DUNG

- 12.1. Khái niệm chỉ mục (index) và vai trò trong tối ưu hóa truy vấn.
- 12.2. Các loại chỉ mục
- 12.3. Giới thiệu lệnh EXPLAIN trong SQL Server
- 12.4. Kỹ thuật viết truy vấn hiệu quả hơn
- 12.5. Khái niệm giao dịch (transaction) trong hệ quản trị cơ sở dữ liệu
- 12.6. Bốn tính chất ACID:

Mục tiêu của bài học

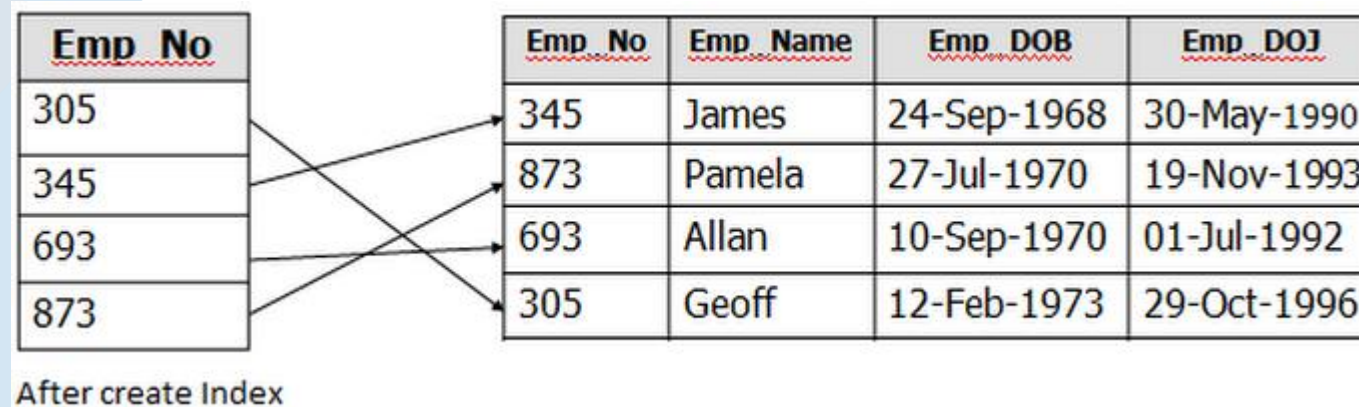
1. Trình bày được khái niệm chỉ mục (index) và vai trò của nó trong việc tăng hiệu suất truy vấn cơ sở dữ liệu.
2. Phân biệt được các loại chỉ mục phổ biến trong hệ quản trị CSDL
3. Sử dụng lệnh EXPLAIN / Execution Plan để phân tích kế hoạch thực thi truy vấn, đánh giá chi phí và chỉ số hiệu năng.
4. Áp dụng các kỹ thuật tối ưu truy vấn SQL
5. Hiểu được mối liên hệ giữa thiết kế truy vấn và hiệu năng hệ thống, từ đó biết cách viết SQL hiệu quả và có thể bảo trì.



12.1. Khái niệm chỉ mục (index) và vai trò trong tối ưu hóa truy vấn

Chỉ mục (Index) là gì?

- **Chỉ mục (Index)** là một cấu trúc dữ liệu đặc biệt, giúp tăng tốc độ truy vấn dữ liệu trong CSDL.
- Tương đương với **mục lục trong sách**: cho phép DBMS nhanh chóng truy cập vị trí dữ liệu mong muốn.
- Có thể hiểu chỉ mục như **bản tra cứu các giá trị của cột**, liên kết với con trỏ đến dòng dữ liệu.



Mục đích và cách hoạt động

- **Mục đích chính:**
 - Tăng tốc truy vấn SELECT, WHERE, JOIN, ORDER BY
 - Tối đa hóa truy vấn trên bảng lớn, giảm thời gian
- **Cách hoạt động:**
 - Chỉ mục là một bản sao được sắp xếp theo 1 hoặc nhiều cột.
 - DBMS duyệt trong chỉ mục thay vì quét toàn bộ bảng.
 - Đóng vai trò "mục lục" giúp truy cập nhanh trang cần thiết.

Ưu điểm và nhược điểm của chỉ mục

- **Ưu điểm:**
 - Tăng tốc truy vấn SQL đối với bảng lớn
 - Hỗ trợ hiệu suất trong SELECT, WHERE, JOIN
 - Tối đa hóa truy vấn trong các hệ thống giao dịch lớn
- **Nhược điểm:**
 - Chậm INSERT, UPDATE, DELETE do phải cập nhật chỉ mục
 - Tốn địa (lưu chỉ mục phụ)
 - Cần xem xét các cột thường lọc trước khi tạo

- Bài toán: Tìm danh sách phân công theo nhân viên

SELECT * FROM PHANCONG WHERE MaNV = 'NV01';

- **Không có index:** Quét toàn bộ bảng (full scan) **Với index trên MaNV:** Truy vấn nhanh hơn nhờ index seek
- Thực tế ứng dụng:
 - Index trên MaSV trong DANGKY
 - Index trên MaKH, MaHD trong hệ thống bán hàng
 - Index trên MaLop, HocKy trong CSDL trường học

Lưu ý khi sử dụng index

- KHÔNG nên tạo chỉ mục cho bảng nhỏ hoặc bảng cập nhật thường xuyên
- KHÔNG nên dùng index trên cột nhiều NULL hoặc thay đổi nhiều
- Nên chọn cột được sử dụng trong WHERE, ORDER BY, JOIN
- Cân nhắc Single vs Composite index dựa vào truy vấn thực tế



ĐẠI HỌC ĐÀ NẴNG

TRƯỜNG ĐẠI HỌC BÁCH KHOA

UNIVERSITY OF SCIENCE AND TECHNOLOGY - UD



12.2. Các loại chỉ mục

THINKING - CREATING - HUMANITY CHERISHING



Các loại chỉ mục

1. Single-Column Index (chỉ mục đơn)

- Chỉ mục trên 1 cột duy nhất
- Dùng khi truy vấn lọc chủ yếu theo 1 cột
- Ví dụ:

```
CREATE INDEX idx_MaNV ON PHANCONG(MaNV);
```

2. Composite Index (chỉ mục tổ hợp)

- Chỉ mục trên nhiều cột
- Tối ưu nếu truy vấn thường lọc theo nhiều cột kết hợp
- Ví dụ:

```
CREATE INDEX idx_NV_DA ON PHANCONG(MaNV, MaDA);
```

3. Unique Index (chỉ mục duy nhất))

- Đảm bảo giá trị không trùng trong cột/các cột được đánh chỉ mục
- Hỗ trợ kiểm tra ràng buộc logic dữ liệu
- Ví dụ:

```
CREATE UNIQUE INDEX idx_Unique_MaNV ON NHANVIEN(MaNV);
```

4. Implicit Index (chỉ mục ngầm định)

- Được tự động tạo ra khi định nghĩa PRIMARY KEY hoặc UNIQUE constraint trên bảng
- Không cần tạo bằng tay, nhưng bạn có thể kiểm tra trong hệ thống:

```
SELECT name, type_desc FROM sys.indexes WHERE object_id = OBJECT_ID('TenBang');
```

- Truy vấn vẫn như bình thường, DBMS sẽ tự dùng chỉ mục nếu phù hợp

5. B-Tree Index (cấu trúc cây cân bằng)

- Là kiểu chỉ mục mặc định trong SQL Server / PostgreSQL
- Hệ thống tự động sử dụng cấu trúc B-tree khi bạn tạo chỉ mục bằng **CREATE INDEX**
- Dùng tốt với truy vấn lọc theo: =, >, <, BETWEEN, ORDER BY
- Không cần chọn B-tree thủ công, **mặc định đã là B-tree**:

6. Hash Index (ít phổ biến hơn)

- Chỉ dùng trong **PostgreSQL**, hoặc các hệ thống hỗ trợ đặc biệt (Memory table)
- Cách hoạt động: dùng **hàm băm** để ánh xạ giá trị → vị trí dòng
- Tốt với truy vấn **so sánh đúng (=)**, **KHÔNG** hỗ trợ > < ORDER BY
- Chỉ nên dùng khi bạn chắc chắn chỉ lọc bằng dấu = và không cần sắp xếp:

Khi nào dùng loại chỉ mục nào?

Loại chỉ mục	Dùng khi...
Single-column	Truy vấn chỉ theo 1 cột duy nhất
Composite index	Truy vấn theo nhiều cột cùng lúc
Unique index	Cần đảm bảo dữ liệu không trùng lặp
B-tree index	Truy vấn có sắp xếp, lọc phạm vi, $\geq$, $\leq$, ORDER BY
Hash index	Truy vấn so sánh đúng bằng (=)
Implicit index	Có ràng buộc PK/UNIQUE tự động sinh ra



12.3 – Phân tích truy vấn với EXPLAIN

Giới thiệu Execution Plan

- Là bản đồ thực thi truy vấn do DBMS lựa chọn
- Giúp hiểu: truy vấn chạy như thế nào
- Quan trọng trong việc tối ưu hóa hiệu năng

Phân tích truy vấn với EXPLAIN

- DBMS có thể chọn nhiều cách thực thi cùng một câu truy vấn SQL.
- EXPLAIN (PostgreSQL) hoặc Execution Plan (SQL Server) giúp hiển thị kế hoạch thực thi mà hệ thống chọn.
- Tác dụng:
 - Xác định DBMS có dùng chỉ mục không (Index Seek / Table Scan)
 - Hiển thị chi phí, số lượng dòng đọc, phương pháp JOIN
 - Giúp phát hiện truy vấn chậm do không tối ưu

Cách xem Execution Plan trong SQL Server

1. Mở SSMS
2. Nhấn Ctrl + M
3. Chạy truy vấn SQL như:

```
SELECT * FROM PHANCONG WHERE MaNV = 'NV01';
```

4. Xem kết quả ở tab Execution Plan

Các thành phần thường gặp

Thành phần	Vai trò
Index Seek	✓ Dùng index, hiệu quả
Table Scan	✗ Quét toàn bảng, chậm
Key Lookup	Lấy thêm cột không nằm trong chỉ mục
Cost %	Ước lượng chi phí tương đối

Ví dụ minh họa

```
CREATE INDEX idx_MaNV ON PHANCONG(MaNV);
```

```
SELECT * FROM PHANCONG WHERE MaNV = 'NV03';
```

- → Execution Plan sẽ xuất hiện node: Index Seek

```
SELECT * FROM PHANCONG WHERE YEAR(GETDATE()) > 2023;
```

- → SQL Server KHÔNG thể tận dụng chỉ mục → dẫn đến Table Scan

Lợi ích khi phân tích kế hoạch thực thi

- Giúp cải thiện tốc độ truy vấn
- Giảm chi phí tài nguyên hệ thống
- Tìm điểm nghẽn hiệu năng
- Hỗ trợ quyết định tạo/chỉnh sửa chỉ mục hợp lý



12.4: Hướng dẫn tối ưu truy vấn trong SQL Server

Hạn chế dùng SELECT ***

- SELECT * truy vấn tất cả các cột trong bảng → Tăng chi phí truy vấn không cần thiết
- Nên chỉ lựa chọn các cột cần thiết:

-- Tốt:

```
SELECT MaNV, MaDA FROM PHANCONG;
```

-- Kém:

```
SELECT * FROM PHANCONG;
```

- Truy vấn rõ ràng cột giúp DBMS tối ưu hơn (dùng Covering Index)

Dùng chỉ mục với khóa tìm kiếm

- Tạo index cho các cột thường xuất hiện trong WHERE, JOIN, ORDER BY

```
CREATE INDEX idx_MaDA ON PHANCONG(MaDA);
```

- Truy vấn:

```
SELECT * FROM PHANCONG WHERE MaDA = 'DA01';
```

- Index giúp truy vấn nhanh hơn nhiều lần so với quét toàn bảng

Tránh truy vấn lồng không cần thiết

- Truy vấn lồng (subquery) nhiều tầng dễ khiến truy vấn chậm
- Nên chuyển sang JOIN nếu có thể:

--Kém:

```
SELECT TenNV FROM NHANVIEN WHERE MaNV IN (  
    SELECT MaNV FROM PHANCONG WHERE SoGio > 10);
```

--Tốt:

```
SELECT DISTINCT NV.TenNV  
FROM NHANVIEN NV JOIN PHANCONG PC ON NV.MaNV = PC.MaNV  
WHERE PC.SoGio > 10;
```

- JOIN giúp tối ưu Execution Plan và dễ tối ưu bởi DBMS

Tránh truy vấn lồng không cần thiết

- Truy vấn lồng (subquery) nhiều tầng dễ khiến truy vấn chậm
- Nên chuyển sang JOIN nếu có thể:

--Kém:

```
SELECT TenNV FROM NHANVIEN WHERE MaNV IN (  
    SELECT MaNV FROM PHANCONG WHERE SoGio > 10);
```

--Tốt:

```
SELECT DISTINCT NV.TenNV  
FROM NHANVIEN NV JOIN PHANCONG PC ON NV.MaNV = PC.MaNV  
WHERE PC.SoGio > 10;
```

- JOIN giúp tối ưu Execution Plan và dễ tối ưu bởi DBMS

12.5. Khái niệm giao dịch (transaction)

Khái niệm giao dịch

- Giao dịch = Một chuỗi thao tác phải được thực hiện toàn bộ hoặc không gì cả
- Dùng để xử lý các tình huống quan trọng như:
 - Giao dịch chuyển tiền
 - Đặt hàng – cập nhật kho – ghi log đơn hàng
- Nếu xảy ra lỗi giữa chừng → cần quay về trạng thái ban đầu
- Ví dụ thực tế – Chuyển tiền

BEGIN TRANSACTION;

UPDATE TAIKHOAN **SET** SoDu = SoDu - 1000000 WHERE **MaKH** = 'KH01';

-- Mất điện ở bước này?

UPDATE TAIKHOAN **SET** SoDu = SoDu + 1000000 WHERE **MaKH** = 'KH02';

COMMIT;

→ Nếu không dùng giao dịch → Dữ liệu sẽ **bị sai lệch nghiêm trọng**

- Ngăn chặn trạng thái dữ liệu không đồng nhất
- Hỗ trợ hệ thống phục hồi khi lỗi (power failure, crash)
- Cần thiết trong mọi hệ thống có nhiều bước cập nhật
- Ví dụ: Cập nhật tồn kho – xuất hàng – trừ tài khoản

Đặc trưng kỹ thuật của giao dịch

- Được quản lý bởi **DBMS transaction Manager**
- Có thể lồng nhau (nested transaction)
- Cần kết hợp với cơ chế kiểm soát đồng thời và phục hồi
- Dùng trong môi trường multi-user để tránh xung đột dữ liệu

Các thao tác trong 1 giao dịch

1. Bắt đầu giao dịch → **BEGIN TRANSACTION**
2. Thực hiện 1 hoặc nhiều lệnh SQL (UPDATE, INSERT...)
3. Nếu thành công: **COMMIT**
4. Nếu lỗi hoặc kiểm tra thất bại: **ROLLBACK**

Ghi nhớ: Dữ liệu **chỉ** được ghi vĩnh viễn sau **COMMIT**

So sánh giao dịch và SQL đơn

SQL đơn	Giao dịch
1 thao tác độc lập	Nhiều thao tác logic
Không kiểm soát lỗi tổng	Có thể rollback toàn bộ nếu lỗi
Không hỗ trợ đồng thời	Có kiểm soát & đồng bộ dữ liệu

• ...

12.6. Bốn tính chất ACID

ACID là gì?

- ACID là 4 tính chất đảm bảo giao dịch được thực hiện an toàn và chính xác
- Viết tắt của:
 - **A**tomicity – Tính nguyên tử
 - **C**onsistency – Tính nhất quán
 - **I**solation – Tính cô lập
 - **D**urability – Tính bền vững

Atomicity – Tính nguyên tử

- Một giao dịch **hoặc thực hiện tất cả**, hoặc **không thực hiện gì cả**
 - Không có trạng thái “thực hiện một phần”
 - Nếu có lỗi → toàn bộ giao dịch bị **ROLLBACK**
-  Ví dụ: Chuyển tiền bị lỗi sau khi đã trừ tiền người gửi → hệ thống tự động hoàn tác

Consistency – Tính nhất quán

- Giao dịch phải **duy trì các ràng buộc** toàn vẹn dữ liệu
- CSDL phải ở trạng thái hợp lệ **trước và sau giao dịch**
- 📌 Ví dụ: Không thể để tài khoản có số dư âm sau giao dịch nếu có ràng buộc kiểm tra

Isolation – Tính cô lập

- Các giao dịch đồng thời **không được ảnh hưởng nhau**
 - Hệ quản trị CSDL đảm bảo mỗi giao dịch thực hiện như thể **đơn lẻ**
-  Ví dụ: Hai người cùng sửa đơn hàng → hệ thống cô lập giao dịch để tránh xung đột

Durability – Tính bền vững

- Khi giao dịch **được COMMIT**, thay đổi phải **được ghi vĩnh viễn**
- Dù mất điện, hệ thống phải khôi phục đúng trạng thái sau COMMIT
- 📌 Áp dụng cơ chế log, ghi ra đĩa... để đảm bảo dữ liệu không bị mất

Tổng kết ACID



- **Atomicity:** tất cả hoặc không gì cả
 - **Consistency:** dữ liệu luôn hợp lệ
 - **Isolation:** tránh xung đột khi chạy song song
 - **Durability:** lưu dữ liệu an toàn sau khi hoàn tất
- ACID = nền tảng đảm bảo **độ tin cậy** và **toàn vẹn** cho hệ thống CSDL

